

Devoir surveillé terminal

Durée 2h00

Aucun document, outil de calcul ou de communication autorisé.

Toute réponse doit être justifiée et chaque étape des algorithmes utilisés clairement spécifiée.

Exercice 1 (Questions de cours - 2 pts)

1°) Quelles sont les différences entre une analyse ascendante et descendante ? Donnez un exemple.

2°) Lors de la génération de quadruplets, il est parfois nécessaire de faire des retours en arrière pour modifier des quadruplets. Expliquez pourquoi et donnez un exemple.

Exercice 2 (Analyseur LL(1) - 8 pts)

Soit la grammaire G suivante :

$$F \rightarrow T \textit{id} (L) ; \quad (1)$$

$$L \rightarrow \epsilon \quad (2)$$

$$L \rightarrow T \textit{id} L' \quad (3)$$

$$L' \rightarrow , T \textit{id} L' \quad (4)$$

$$L' \rightarrow \epsilon \quad (5)$$

$$T \rightarrow \textit{int} \quad (6)$$

$$T \rightarrow \textit{double} \quad (7)$$

1°) Donnez la définition formelle de cette grammaire.

2°) Construisez les ensembles *Premier* et *Suivant* des symboles non terminaux.

3°) Cette grammaire est-elle LL(1) ?

4°) Construisez la TASD.

5°) Exécutez l'analyseur sur le mot suivant (précisez la cible, le texte source et les actions) :

`int toto(int a, double b);`

Exercice 3 (Analyseur LR(1) - 6 pts)

Soit la grammaire suivante :

$$G \rightarrow R \quad (1)$$

$$G \rightarrow R ; G \quad (2)$$

$$R \rightarrow \textit{id} : L \quad (3)$$

$$L \rightarrow \textit{id} L \quad (4)$$

$$L \rightarrow \epsilon \quad (5)$$

1°) Construisez les ensembles *Premier* et *Suivant* des symboles non terminaux.

2°) Construisez les ensembles d'items LR(1).

3°) Construisez la table d'analyse selon la méthode LR(1).

4°) **[bonus]** Est-il possible de construire un analyseur LALR(1) ?

Exercice 4 (Décoration de grammaire - 4 pts)

Nous souhaitons pouvoir créer des arbres d'évaluation typés (des arbres syntaxiques représentant une expression arithmétique) à partir de la grammaire suivante :

$$\begin{aligned} E &\rightarrow E_1 + T \mid T \\ T &\rightarrow T_1 * F \mid F \\ F &\rightarrow id \mid entier \mid reel \mid (E) \end{aligned}$$

Nous considérons les fonctions suivantes (la table des symboles est considérée existante et accessible à tout endroit ; elle constituée d'éléments de type `entree_t`) :

```
/**
 * Création d'une feuille.
 * @param type ID ou CST
 * @param valeur un entier, un réel ou un entree_t
 * @return un noeud
 */
creer_feuille(type, valeur) : noeud_t

/**
 * Création d'un noeud.
 * @param type opération ADD ou MUL ou conversion CONV_INT ou CONV_DOUBLE
 * @param noeud1 le premier argument
 * @param noeud2 le deuxième argument (NULL pour CONV_INT et CONV_DOUBLE)
 * @return un noeud
 */
creer_noeud(type, noeud1, noeud2) : noeud_t

/**
 * Retourne le type d'une entrée de la table des symboles.
 * @param str le nom de l'entrée
 * @return le type de l'entrée (constante INT ou DOUBLE)
 */
type_entree(str) : int

/**
 * Retourne l'entrée de la table des symboles.
 * @param str le nom de l'entrée
 * @return l'entrée
 */
entree(str) : entree_t
```

1°) Quels sont les attributs nécessaires pour chaque symbole de la grammaire ?

2°) Proposez une décoration de la grammaire permettant de créer les arbres d'évaluation.